

у меня сейчас выполняется такой проект, шаг 1.4  
успешно выполнен давай перейдем к шагу 1.5

Отлично, тогда двигаемся дальше — на шаге **1.5** твоя задача уже не про внешний вид, а про **то, чтобы кнопки реально переключали экраны**.

Сейчас у тебя уже есть:

- `QStackedWidget` (из 1.3)
- `MainMenu` с кнопками (из 1.4)

Теперь делаем **связку кнопок → смена экрана**.

---

## Шаг 1. Добавь новые экраны в MainWindow

Открой `ui/main_window.py` и добавь заготовки экранов:

```
from PyQt6.QtWidgets import QMainWindow, QStackedWidget, QLabel

from ui.main_menu import MainMenu


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Yoga Practice App")
        self.setGeometry(200, 200, 900, 600)

        self.stack = QStackedWidget()
        self.setCentralWidget(self.stack)

        # === ЭКРАНЫ ===
        self.main_menu = MainMenu(self)

        self.asanas_screen = QLabel("Экран: Асаны")
```

```
self.practice_screen = QLabel("Экран: Практика")
self.music_screen = QLabel("Экран: Музыка")

# === ДОБАВЛЯЕМ В STACK ===
self.stack.addWidget(self.main_menu)
self.stack.addWidget(self.asanas_screen)
self.stack.addWidget(self.practice_screen)
self.stack.addWidget(self.music_screen)

# === МЕТОДЫ ПЕРЕКЛЮЧЕНИЯ ===
def show_main_menu(self):
    self.stack.setCurrentWidget(self.main_menu)

def show_asanas(self):
    self.stack.setCurrentWidget(self.asanas_screen)

def show_practice(self):
    self.stack.setCurrentWidget(self.practice_screen)

def show_music(self):
    self.stack.setCurrentWidget(self.music_screen)
```



## Шаг 2. Подключи кнопки в MainMenu

Теперь открой `ui/main_menu.py` .

Допустим, у тебя есть кнопки:

```
self.btn_asanas
self.btn_practice
self.btn_music
```

Добавь обработчики клика:

```
self.btn_asanas.clicked.connect(self.main_window.show_asanas)
self.btn_practice.clicked.connect(self.main_window.show_practice)
self.btn_music.clicked.connect(self.main_window.show_music)
```



Важно: MainMenu должен принимать main\_window :

```
class MainMenu(QWidget):
    def __init__(self, main_window):
        super().__init__()
        self.main_window = main_window
```

## ✓ Что должно получиться

Теперь:

- нажимаешь **Асаны** → открывается экран "Экран: Асаны"
- нажимаешь **Практика** → другой экран
- навигация реально работает

## 👉 Мини-проверка

Если не работает, проверь:

- передаёшь ли `self` в `MainMenu(self)`
- совпадают ли имена кнопок
- нет ли ошибки типа `NoneType`

## 🚀 Дальше (шаг 1.6)

Когда это заработает, следующий логичный шаг:

👉 добавить кнопку "назад" на каждом экране

Если хочешь, скинь свой `main_menu.py` — я прямо по коду покажу, куда вставить `connect`, без угадываний.

ошибка

Пока слишком мало информации — "ошибка" может быть где угодно 😊

Давай быстро сузим проблему. Пришли одно из этого:

1. Текст ошибки полностью (traceback из консоли)
2. Или скрин

### 3. Или куски кода:

- `main_window.py`
- `main_menu.py`

---

А пока — вот самые частые причины именно на шаге 1.5:

#### ! 1. MainMenu не принимает main\_window

Проверь:

```
class MainMenu(QWidget):
    def __init__(self, main_window):
        super().__init__()
        self.main_window = main_window
```

#### ! 2. Передаёшь не self

В MainWindow должно быть:

```
self.main_menu = MainMenu(self)
```

#### ! 3. Ошибка типа:

```
AttributeError: 'MainMenu' object has no attribute 'main_window'
```



→ значит ты не сохранил `self.main_window`

#### ! 4. Ошибка типа:

```
AttributeError: 'MainWindow' object has no attribute 'show_asanas'
```



→ значит метод не добавлен или опечатка

---

## ! 5. Кнопка называется иначе

Например:

```
self.btn_asana    # а не btn_asanas
```

Скинь ошибку — я сразу скажу точную причину и исправим за минуту 👍

```
{
  "resource":
    "/home/julia7/YogaProject/ui/main_menu.py",
    "owner": "Pylance3",
    "code": {
      "value": "reportUndefinedVariable",
      "target": {
        "$mid": 1,
        "path": "/microsoft/pylance-
release/blob/main/docs/diagnostics/reportUndefinedVaria
ble.md",
        "scheme": "https",
        "authority": "github.com"
      }
    },
    "severity": 4,
    "message": "\"main_window\" не определено",
    "source": "Pylance",
    "startLineNumber": 11,
    "startColumn": 28,
    "endLineNumber": 11,
    "endColumn": 39,
    "modelVersionId": 43,
    "origin": "extHost2"
}
```

Отлично, это уже конкретика 👍

Ошибка понятная и очень типичная:

"main\_window не определено"

Это значит, что ты используешь `main_window`, но не передаёшь его в класс.

---

## В чём проблема

Скорее всего у тебя сейчас что-то вроде:

```
class MainMenu(QWidget):
    def __init__(self):
        super().__init__()

        self.btn_asanas.clicked.connect(main_window.show_asanas) # ❌
```

👉 Переменная `main_window` тут просто не существует.

---

## Как правильно исправить

### 1. Передаём `main_window` в конструктор

```
class MainMenu(QWidget):
    def __init__(self, main_window):
        super().__init__()

        self.main_window = main_window
```

### 2. Используем `self.main_window`

```
self.btn_asanas.clicked.connect(self.main_window.show_asanas)
self.btn_practice.clicked.connect(self.main_window.show_practice)
self.btn_music.clicked.connect(self.main_window.show_music)
```

---

### 3. Проверяем создание в MainWindow

В `main_window.py` ОБЯЗАТЕЛЬНО:

```
self.main_menu = MainMenu(self)
```



### Почему это работает

Ты фактически делаешь так:

- `MainWindow` создаёт `MainMenu`
- передаёт себя ( `self` )
- `MainMenu` теперь знает, кто главный и может вызывать его методы



### Быстрая проверка

Если после этого появится другая ошибка — она, скорее всего, будет одной из этих:

- `show_asanas` не существует → забыли метод
- кнопка называется иначе → опечатка

Если хочешь — скинь свой `main_menu.py`, я прям построчно покажу, что исправить.

вот ошибка Traceback (most recent call last):

File "/home/julia7/YogaProject/main.py", line 16, in

<module>

main()

File "/home/julia7/YogaProject/main.py", line 9, in main

window = MainWindow()

File "/home/julia7/YogaProject/ui/main\_window.py", line 17, in `__init__`

self.main\_menu = MainMenu(self)

File "/home/julia7/YogaProject/ui/main\_menu.py", line 48, in `__init__`

self.btn\_asanas.clicked.connect(self.main\_window.show\_asanas)

```
AttributeError: 'MainMenu' object has no attribute
'btn_asanas' вот код from PyQt6.QtWidgets import
QWidget, QVBoxLayout, QPushButton, QLabel
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QPixmap
import os
```

```
class MainMenu(QWidget):
    def __init__(self, main_window):
        super().__init__()

        self.main_window = main_window

        self.setMinimumSize(800, 600)

        # === ФОН ===
        self.bg_label = QLabel(self)
        self.bg_label.setGeometry(0, 0, 800, 600)
        self.bg_label.setScaledContents(True)

        base_dir =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        image_path = os.path.join(base_dir, "assets", "images",
"background.jpg")

        pixmap = QPixmap(image_path)
        self.bg_label.setPixmap(pixmap)

        # === КОНТЕНТ ===
        self.container = QWidget(self)
        self.container.setGeometry(0, 0, 800, 600)

        layout = QVBoxLayout()
        layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
        layout.setSpacing(25)

        # Заголовок
        title = QLabel("Главное меню")
        title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        title.setStyleSheet("""
            font-size: 40px;
```



```
color: white;
font-weight: bold;
""")
```

```
# Кнопки
```

```
practice_btn = QPushButton("Практика")
```

```
asanas_btn = QPushButton("Асаны")
```

```
music_btn = QPushButton("Музыка")
```

```
self.btn_asanas.clicked.connect(self.main_window.show_a
sanas)
```

```
self.btn_practice.clicked.connect(self.main_window.show_
practice)
```

```
self.btn_music.clicked.connect(self.main_window.show_m
usic)
```

```
button_style = """
QPushButton {
    border-radius: 25px;
    padding: 15px;
    font-size: 18px;
    min-width: 250px;
    background-color: rgba(255, 255, 255, 0.85);
    color: #333;
}
QPushButton:hover {
    background-color: rgba(255, 255, 255, 1);
}
"""
```

```
practice_btn.setStyleSheet(button_style)
```

```
asanas_btn.setStyleSheet(button_style)
```

```
music_btn.setStyleSheet(button_style)
```

```
layout.addWidget(title)
```

```
layout.addWidget(practice_btn)
```

```
layout.addWidget(asanas_btn)
```

```
layout.addWidget(music_btn)
```

```
self.container.setLayout(layout)
```

```
def resizeEvent(self, event):
    # растягиваем фон при изменении окна
    self.bg_label.setGeometry(0, 0, self.width(),
self.height())
    self.container.setGeometry(0, 0, self.width(),
self.height())
```

Ошибка у тебя сейчас максимально конкретная — и она прямо в коде:

```
AttributeError: 'MainMenu' object has no attribute 'btn_asanas'
```

---

## В чём проблема

Ты создал кнопки как **локальные переменные**:

```
practice_btn = QPushButton("Практика")
asanas_btn = QPushButton("Асаны")
music_btn = QPushButton("Музыка")
```

Но потом пытаешься обратиться к ним как к **полям объекта**:

```
self.btn_asanas.clicked.connect(...) # ❌ такого нет
```

👉 `self.btn_asanas` просто не существует.

---

## Как исправить (2 варианта)

✓ **Вариант 1 (лучший)** — сделать кнопки через `self`

Замени:

```
practice_btn = QPushButton("Практика")
asanas_btn = QPushButton("Асаны")
music_btn = QPushButton("Музыка")
```

на:

```
self.btn_practice = QPushButton("Практика")
self.btn_asanas = QPushButton("Асаны")
self.btn_music = QPushButton("Музыка")
```

---

И дальше всё заработает:

```
self.btn_asanas.clicked.connect(self.main_window.show_asanas)
self.btn_practice.clicked.connect(self.main_window.show_practice)
self.btn_music.clicked.connect(self.main_window.show_music)
```

---

И не забудь поменять добавление в layout:

```
layout.addWidget(self.btn_practice)
layout.addWidget(self.btn_asanas)
layout.addWidget(self.btn_music)
```

---

## ✓ Вариант 2 (быстрый, но хуже)

Можно оставить как есть и просто:

```
asanas_btn.clicked.connect(self.main_window.show_asanas)
practice_btn.clicked.connect(self.main_window.show_practice)
music_btn.clicked.connect(self.main_window.show_music)
```

👉 Но тогда ты не сможешь удобно управлять кнопками позже (поэтому лучше вариант 1).

---



## Почему это важно

- `self.xxx` → живёт внутри объекта
- `xxx` → просто временная переменная

Ты сейчас смешал эти два подхода — отсюда ошибка.

---

## Дальше

После фикса у тебя:

- кнопки будут нажиматься
- экраны будут переключаться

Если вдруг следующая ошибка появится — кидай, разберём дальше 👍

все работает, давай дальше.

Отлично, теперь у тебя уже есть **рабочая навигация между экранами** — это ключевой момент 🔥

Переходим к следующему шагу.

---

## Шаг 1.6 — кнопка «Назад» на экранах

Сейчас у тебя экраны открываются, но **вернуться назад нельзя**. Исправляем это.

---

### 1. Заменим QLabel на нормальные экраны

В `main_window.py` у тебя сейчас:

```
self.asanas_screen = QLabel("Экран: Асаны")
```

👉 Это временно. Теперь сделаем полноценные виджеты.

---

### ✨ Пример: экран "Асаны"

Добавь новый файл:

`ui/asanas_screen.py`

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QPushButton, QLabel
from PyQt6.QtCore import Qt

class AsanasScreen(QWidget):
    def __init__(self, main_window):
        super().__init__()

        self.main_window = main_window

        layout = QVBoxLayout()
        layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
        layout.setSpacing(20)

        title = QLabel("Асаны")
        title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        title.setStyleSheet("font-size: 30px;")

        back_btn = QPushButton("← Назад")
        back_btn.clicked.connect(self.main_window.show_main_menu)

        layout.addWidget(title)
        layout.addWidget(back_btn)

        self.setLayout(layout)
```

---

## 2. Подключаем в MainWindow

В main\_window.py :

```
from ui.asanas_screen import AsanasScreen
```

И заменяем:

```
self.asanas_screen = AsanasScreen(self)
```

---

## 3. Повтори для других экранов

Создай:

- `practice_screen.py`
- `music_screen.py`

По той же логике:

- заголовок
- кнопка "Назад"

---

## ✅ Что получится

Теперь:

- нажал "Асаны" → открылся экран
- нажал "← Назад" → вернулся в меню

👉 у тебя появляется полноценная навигация как в приложении

---

## 💡 Мини-улучшение (по желанию)

Можно сделать кнопку назад красивее:

```
back_btn.setStyleSheet("""
    QPushButton {
        font-size: 16px;
        padding: 10px;
    }
    """)
```

---

## 🚀 Дальше (шаг 1.7)

Следующий логичный шаг:

👉 сделать экран "Асаны" не пустым, а со списком асан

(например: карточки, список, скролл)

---

Если хочешь — на следующем шаге можем сразу сделать:

👉 красивый список асан с изображениями (это уже выглядит как реальное приложение)

```
from PyQt6.QtWidgets import QMainWindow,  
QStackedWidget, QLabel
```

```
from ui.main_menu import MainMenu
```

```
class MainWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()  
  
        self.setWindowTitle("Yoga Practice App")  
        self.setGeometry(200, 200, 900, 600)  
  
        self.stack = QStackedWidget()  
        self.setCentralWidget(self.stack)  
  
        # === ЭКРАНЫ ===  
        self.main_menu = MainMenu(self)  
        self.asanas_screen = AsanasScreen(self)  
  
        self.asanas_screen = QLabel("Экран: Асаны")  
        self.practice_screen = QLabel("Экран: Практика")  
        self.music_screen = QLabel("Экран: Музыка")  
  
        # === ДОБАВЛЯЕМ В STACK ===  
        self.stack.addWidget(self.main_menu)  
        self.stack.addWidget(self.asanas_screen)  
        self.stack.addWidget(self.practice_screen)  
        self.stack.addWidget(self.music_screen)  
  
        # === МЕТОДЫ ПЕРЕКЛЮЧЕНИЯ ===  
        def show_main_menu(self):  
            self.stack.setCurrentWidget(self.main_menu)  
  
        def show_asanas(self):  
            self.stack.setCurrentWidget(self.asanas_screen)  
  
        def show_practice(self):  
            self.stack.setCurrentWidget(self.practice_screen)  
  
        def show_music(self):
```

```
self.stack.setCurrentWidget(self.music_screen)
ошибка [{
  "resource":
"/home/julia7/YogaProject/ui/main_window.py",
  "owner": "Pylance3",
  "code": {
    "value": "reportUndefinedVariable",
    "target": {
      "$mid": 1,
      "path": "/microsoft/pylance-
release/blob/main/docs/diagnostics/reportUndefinedVaria
ble.md",
      "scheme": "https",
      "authority": "github.com"
    }
  },
  "severity": 4,
  "message": "\"AsanasScreen\" не определено",
  "source": "Pylance",
  "startLineNumber": 18,
  "startColumn": 30,
  "endLineNumber": 18,
  "endColumn": 42,
  "modelVersionId": 17,
  "origin": "extHost2"
}]
```

У тебя тут сразу **две проблемы**, и одна из них даже важнее ошибки Pylance.

---

## Проблема 1: AsanasScreen не импортирован

Ошибка:

```
"AsanasScreen" не определено
```

 Ты используешь класс, но **не импортировал его**.

---

## Исправление

В начале файла `main_window.py` добавь:



```
from ui.asanas_screen import AsanasScreen
```

## ❌ Проблема 2 (критичнее): ты перезаписываешь экран

Вот это у тебя сейчас:

```
self.asanas_screen = AsanasScreen(self)

self.asanas_screen = QLabel("Экран: Асаны") # ❌ ПЕРЕЗАТЕП
```

👉 Ты создал нормальный экран... и тут же уничтожил его 😊

## ✅ Исправление

Удаляешь эту строку:

```
self.asanas_screen = QLabel("Экран: Асаны")
```

## ✅ Итоговый правильный код

```
from PyQt6.QtWidgets import QMainWindow, QStackedWidget, QLabel

from ui.main_menu import MainMenu
from ui.asanas_screen import AsanasScreen


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Yoga Practice App")
        self.setGeometry(200, 200, 900, 600)
```

```
self.stack = QStackedWidget()
self.setCentralWidget(self.stack)

# === ЭКРАНЫ ===
self.main_menu = MainMenu(self)
self.asanas_screen = AsanasScreen(self)

self.practice_screen = QLabel("Экран: Практика")
self.music_screen = QLabel("Экран: Музыка")

# === STACK ===
self.stack.addWidget(self.main_menu)
self.stack.addWidget(self.asanas_screen)
self.stack.addWidget(self.practice_screen)
self.stack.addWidget(self.music_screen)
```



## Важный принцип

Если видишь что-то типа:

```
self.x = ...
self.x = ...
```

👉 почти всегда это баг (перезапись)



## Дальше

Теперь у тебя:

- экран "Асаны" — настоящий
- кнопка "назад" будет работать

Следующий шаг логичный:

👉 сделать **practice\_screen** и **music\_screen** такими же классами, а не QLabel

Хочешь — ↪ могу сразу дать шаблон, чтобы ты просто копирастнул и ускоришься.

python main.py

Traceback (most recent call last):

File "/home/julia7/YogaProject/main.py", line 16, in

```
<module>
  main()
File "/home/julia7/YogaProject/main.py", line 9, in main
  window = MainWindow()
File "/home/julia7/YogaProject/ui/main_window.py", line
18, in __init__
  self.main_menu = MainMenu(self)
File "/home/julia7/YogaProject/ui/main_menu.py", line 48,
in __init__

self.btn_asanas.clicked.connect(self.main_window.show_a
sanas)
AttributeError: 'MainWindow' object has no attribute
'show_asanas'
```

Отлично, ты почти у цели — это уже **последний типичный баг в этой цепочке**.

---

## ❌ Ошибка

AttributeError: 'MainWindow' object has no attribute 'show\_asanas'

👉 Кнопка пытается вызвать метод, которого нет (или Python его не видит).

---

## 🔍 Причины (почти всегда одна из этих)

### 1. ❌ Метод написан с отступом не внутри класса

Очень частая ошибка.

Проверь, чтобы у тебя было **внутри класса** MainWindow:

```
class MainWindow(QMainWindow):
    def __init__(self):
        ...

    def show_asanas(self):
        self.stack.setCurrentWidget(self.asanas_screen)
```

👉 Если он выглядит так:

```
def show_asanas(self): # ❌ вне класса
    ...
```

— Python его не видит как метод класса.

---

## 2. ❌ Опечатка в названии

Например:

```
def show_asana(self): # нет "s"
```

а вызываешь:

```
show_asanas
```

---

## 3. ❌ Метод удалился при правке

Ты раньше его писал, но мог случайно стереть.

---



## Быстрая проверка (сейчас сделай)

В `main_window.py` должно быть ВСЁ это:

```
def show_main_menu(self):
    self.stack.setCurrentWidget(self.main_menu)

def show_asanas(self):
    self.stack.setCurrentWidget(self.asanas_screen)

def show_practice(self):
    self.stack.setCurrentWidget(self.practice_screen)

def show_music(self):
    self.stack.setCurrentWidget(self.music_screen)
```



## Маленький лайфхак

Если сомневаешься — вставь временно:

```
print(dir(self))
```

в `MainMenu` перед `connect` — и увидишь, есть ли `show_asanas` .

---



## Скорее всего у тебя проблема №1 (отступ)

Это самая частая причина именно такой ошибки.

---

Если не работает — скинь `main_window.py` , я за 10 секунд найду, где косяк.

```
from PyQt6.QtWidgets import QMainWindow,  
QStackedWidget, QLabel
```

```
from ui.main_menu import MainMenu  
from ui.asanas_screen import AsanasScreen
```

```
class MainWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()  
  
        self.setWindowTitle("Yoga Practice App")  
        self.setGeometry(200, 200, 900, 600)  
  
        self.stack = QStackedWidget()  
        self.setCentralWidget(self.stack)  
  
        # === ЭКРАНЫ ===  
        self.main_menu = MainMenu(self)  
        self.asanas_screen = AsanasScreen(self)
```

```
self.practice_screen = QLabel("Экран: Практика")
self.music_screen = QLabel("Экран: Музыка")

# === STACK ===
self.stack.addWidget(self.main_menu)
self.stack.addWidget(self.asanas_screen)
self.stack.addWidget(self.practice_screen)
self.stack.addWidget(self.music_screen) вот mai
window пришли готовый и рабочий код
```

У тебя просто не хватает методов переключения — поэтому и ошибка.

Я дам тебе полностью готовый и чистый `main_window.py`, просто замени файл.

## ✓ Готовый рабочий `main_window.py`

```
from PyQt6.QtWidgets import QMainWindow, QStackedWidget, QLabel

from ui.main_menu import MainMenu
from ui.asanas_screen import AsanasScreen

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Yoga Practice App")
        self.setGeometry(200, 200, 900, 600)

        self.stack = QStackedWidget()
        self.setCentralWidget(self.stack)

        # === ЭКРАНЫ ===
        self.main_menu = MainMenu(self)
        self.asanas_screen = AsanasScreen(self)

        self.practice_screen = QLabel("Экран: Практика")
        self.music_screen = QLabel("Экран: Музыка")

        # === STACK ===
        self.stack.addWidget(self.main_menu)
        self.stack.addWidget(self.asanas_screen)
        self.stack.addWidget(self.practice_screen)
        self.stack.addWidget(self.music_screen)
```

```
# === ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ ===
def show_main_menu(self):
    self.stack.setCurrentWidget(self.main_menu)

def show_asanas(self):
    self.stack.setCurrentWidget(self.asanas_screen)

def show_practice(self):
    self.stack.setCurrentWidget(self.practice_screen)

def show_music(self):
    self.stack.setCurrentWidget(self.music_screen)
```

## 🔥 Что у тебя было не так

Ты:

- создал экраны ✓
- добавил их в stack ✓
- но не добавил методы show\_\* ✗

👉 поэтому кнопки не находили show\_asanas

## ✓ После этого

Теперь у тебя:

- кнопки работают
- переходы работают
- назад (если сделал) тоже работает

## 🚀 Следующий шаг (рекомендую)

Сейчас у тебя:

```
QLabel("Экран: Практика")
```

👉 это временно

Лучше сделать:

- PracticeScreen
- MusicScreen

по тому же шаблону, что и AsanasScreen

---

Если хочешь — могу дать шаблон сразу на 2 экрана, чтобы ты сделал за 2 минуты и проект стал выглядеть как настоящее приложение.

да напиши теперь то же самое для практики и музыки  
и подключи к остальным файлам, пришли готовые  
файлы с исправлениями

Супер, давай сразу сделаем **чисто и правильно**, чтобы у тебя всё было как в нормальном приложении. Ниже — **готовые файлы**, просто создаёшь/заменяешь и запускаешь.

---



## ui/practice\_screen.py

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QPushButton, QLabel
from PyQt6.QtCore import Qt

class PracticeScreen(QWidget):
    def __init__(self, main_window):
        super().__init__()

        self.main_window = main_window

        layout = QVBoxLayout()
        layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
        layout.setSpacing(20)

        title = QLabel("Практика")
        title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        title.setStyleSheet("font-size: 30px;")

        back_btn = QPushButton("← Назад")
        back_btn.clicked.connect(self.main_window.show_main_menu)
```



```
layout.addWidget(title)
layout.addWidget(back_btn)

self.setLayout(layout)
```



## ui/music\_screen.py

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QPushButton, QLabel
from PyQt6.QtCore import Qt

class MusicScreen(QWidget):
    def __init__(self, main_window):
        super().__init__()

        self.main_window = main_window

        layout = QVBoxLayout()
        layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
        layout.setSpacing(20)

        title = QLabel("Музыка")
        title.setAlignment(Qt.AlignmentFlag.AlignCenter)
        title.setStyleSheet("font-size: 30px;")

        back_btn = QPushButton("← Назад")
        back_btn.clicked.connect(self.main_window.show_main_menu)

        layout.addWidget(title)
        layout.addWidget(back_btn)

        self.setLayout(layout)
```



## ОБНОВЛЁННЫЙ ui/main\_window.py

```
from PyQt6.QtWidgets import QMainWindow, QStackedWidget

from ui.main_menu import MainMenu
from ui.asanas_screen import AsanasScreen
from ui.practice_screen import PracticeScreen
from ui.music_screen import MusicScreen
```

```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Yoga Practice App")
        self.setGeometry(200, 200, 900, 600)

        self.stack = QStackedWidget()
        self.setCentralWidget(self.stack)

        # === ЭКРАНЫ ===
        self.main_menu = MainMenu(self)
        self.asanas_screen = AsanasScreen(self)
        self.practice_screen = PracticeScreen(self)
        self.music_screen = MusicScreen(self)

        # === STACK ===
        self.stack.addWidget(self.main_menu)
        self.stack.addWidget(self.asanas_screen)
        self.stack.addWidget(self.practice_screen)
        self.stack.addWidget(self.music_screen)

        # === ПЕРЕКЛЮЧЕНИЕ ===
        def show_main_menu(self):
            self.stack.setCurrentWidget(self.main_menu)

        def show_asanas(self):
            self.stack.setCurrentWidget(self.asanas_screen)

        def show_practice(self):
            self.stack.setCurrentWidget(self.practice_screen)

        def show_music(self):
            self.stack.setCurrentWidget(self.music_screen)

```

## ⚠ Проверь main\_menu.py

У тебя уже должно быть (если делал раньше правильно):

Python



▶ Запустить

```

self.btn_asanas.clicked.connect(self.main_window.show_asanas)
self.btn_practice.clicked.connect(self.main_window.show_practice)
self.btn_music.clicked.connect(self.main_window.show_music)

```



## Что теперь работает

- кнопки меню → открывают экраны
  - на каждом экране есть ← **Назад**
  - структура уже как у реального приложения
- 



## Следующий шаг (очень важный)

Теперь можно делать **наполнение**:



экран "Асаны":

- список асан
- картинки
- карточки



экран "Практика":

- таймер / последовательности



экран "Музыка":

- список треков / звуков
- 

Если хочешь, ↳ следующий шаг можем сделать **красивый список асан с изображениями** — это уже будет выглядеть как готовое приложение.